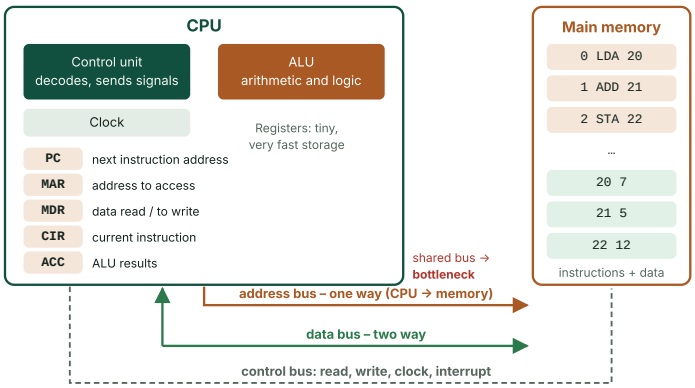
Internal components: CPU, registers, buses, I/O controllers and the Von Neumann architecture

FD08 FACT FILE

Specification: students should be able to

- Describe the internal components of a computer system: processor, clock, main memory, buses, I/O controllers, registers and ports
- Describe the tasks performed by the control unit, the arithmetic logic unit (ALU) and the registers
- Describe the purpose of the program counter (PC), memory address register (MAR), memory data register (MDR), current instruction register (CIR) and accumulator (ACC)
- Describe the function and direction of the address, data and control buses
- Calculate the maximum addressable memory, or an unknown bus width, from the address bus width, data bus width and memory size
- Explain why peripherals are connected through I/O controllers and ports
- Describe the Von Neumann (stored-program) architecture and its main limitation

Key words: central processing unit (CPU), control unit (CU), arithmetic logic unit (ALU), register, program counter (PC), memory address register (MAR), memory data register (MDR), current instruction register (CIR), accumulator (ACC), system bus, address bus, data bus, control bus, bus width, I/O controller, port, motherboard, Von Neumann architecture



The Von Neumann architecture: a CPU containing the control unit, ALU, clock and registers, connected to a single main memory holding instructions and data by the address, data and control buses

How a computer is organised

Almost every general-purpose computer follows the stored-program design set out by John von Neumann in 1945. Program instructions and the data they work on are both held, as binary, in the same main memory. A single processor fetches instructions from that memory one at a time and carries them out. Input devices supply data, output devices present results, and secondary storage keeps programs and files when the power is switched off. All of these parts are linked by the system bus, a set of parallel connections divided into address, data and control lines. Physically, the main components sit on the motherboard, a large printed circuit board with a socket for the processor, slots for memory modules, expansion slots for extra cards such as graphics or network cards, and connectors for drives and external ports. Because instructions are just data held in memory, the same computer can switch from a spreadsheet to a game simply by loading different instructions, which is what makes it general purpose. The cost of this flexibility is that instructions and data must take turns on the same bus, a limit known as the Von Neumann bottleneck.

The control unit, ALU and clock

The central processing unit is made up of the control unit, the arithmetic logic unit and a set of registers, kept in step by a clock signal. The control unit has three main jobs. It manages the fetch–execute cycle, fetching each instruction from memory, decoding it and directing its execution, calling on the ALU whenever a calculation is needed. It sends control signals that coordinate and monitor the rest of the hardware, so that the commands issued by software are carried out in the correct order. It also manages the flow of data into and out of the processor, including input and output. The ALU does the actual processing. It takes values from registers, performs whichever operation the control unit has selected and stores the result in a register, usually the accumulator. Its operations are arithmetic, such as addition and subtraction, and logical, such as AND, OR and NOT, together with comparisons like greater than or equal to, which are what allow programs to make decisions and repeat. The clock produces a regular stream of electrical pulses, and every step the processor takes is timed to them, which is why clock speed is so closely linked to performance.

Registers and what they hold

A register is a single, very small storage location built into the processor. Registers are the fastest storage in the whole computer, and data must be placed in a register before the ALU can work on it: to add two numbers, each is loaded into a register, the ALU adds them and the answer is written back to a register. A register can hold a data value, an instruction or the address of a memory location. General-purpose registers are used freely by programs, while special-purpose registers each have a fixed role in the fetch–execute cycle. The program counter holds the address of the next instruction to be fetched and is incremented after each fetch. The memory address register holds the address of the location about to be read or written. The memory data register is a two-way buffer between processor and memory, holding whatever has just been read or is waiting to be written. The current instruction register holds the instruction being decoded and executed, and the accumulator holds the results of ALU operations. Programmers working in a high-level language such as C# never handle registers themselves; the compiler produces machine code that does.

The system bus

A bus is a group of parallel wires, or tracks on the motherboard, that carries electrical signals between components. Each wire carries one bit at a time, so the number of wires, called the bus width, decides how many bits travel together. The system bus has three parts. The address bus identifies which memory location or I/O device the processor wants to use. It is unidirectional, because only the processor chooses which location to access, so addresses travel outwards from the CPU. The data bus carries the data itself, including instructions, between the processor, memory and I/O devices. It is bidirectional because data is both read into and written out of the processor. The control bus carries command and timing signals such as memory read, memory write, the clock pulse, interrupt requests and bus requests. Each individual control line carries one kind of signal in one direction, but the control bus as a whole is bidirectional because some signals go out from the CPU while others, such as an interrupt from a keyboard or a job-finished message from a printer, come back in. A wider data bus moves more bits per transfer, which improves performance.

Calculating addressable memory

The width of the address bus fixes how many different locations the processor can identify. With n address lines there are 2^n different binary patterns, so 2^n locations numbered from 0 to $2^n - 1$, and each extra line doubles the total. Each location holds as many bits as the data bus carries in one transfer, so the maximum memory in bits is 2^n multiplied by the data bus width. To convert, divide by 8 (2^3) for bytes and then by 2^{10}, 2^{20} or 2^{30} for kilobytes, megabytes or gigabytes, since this course measures memory in binary multiples. Writing every value as a power of two keeps the arithmetic simple, because multiplying means adding indices and dividing means subtracting them. For example, a 24-bit address bus with a 16-bit data bus gives $2^{24} \times 2^4 = 2^{28}$ bits, which is 2^{25} bytes or 32 MB. The same formula can be rearranged to find a missing bus width when the memory size is known. In practice few computers are fitted with all the memory their address bus allows: memory costs money, most software does not need that much, and some addresses are reserved for I/O ports rather than RAM.

I/O controllers and ports

Everything outside the processor, main memory and system bus is a peripheral: an input device, an output device, or a device that does both, such as an external drive. Peripherals work at very different speeds and voltages and use different command formats, so it would be impractical to design a processor able to drive every one directly. Instead each device is connected through an I/O controller, an electronic circuit that sits between the system bus and the device. It contains an interface to the system bus, electronics that exchange signals with the device, and an I/O port through which data passes. The controller converts between the protocol the device uses and the one the computer uses, changes voltage levels to suit the computer, buffers data so that a fast processor is not held up by a slow device, and converts data formats, for example turning a 16-bit value from a device into the 32-bit form the computer expects. Controllers may be built into the motherboard or added on expansion cards. A port is the physical socket where a cable connects. USB has largely replaced the many dedicated ports of older PCs, carrying both data and power through one standard connector.

T Key terms from the mark schemes

Arithmetic logic unit (ALU)
The part of the CPU that performs arithmetic operations, logical operations and comparisons on data held in registers.

Control unit (CU)
The part of the CPU that fetches and decodes instructions and sends control signals to coordinate the processor, memory and I/O.
`Credited: Coordinates the actions of the computer / manages the fetch–execute cycle`

Register
A single, very fast storage location inside the CPU that holds data, an instruction or an address during processing.

Program counter (PC)
A register that holds the address of the next instruction to be fetched; it is incremented after each fetch.

Memory address register (MAR)
A register that holds the address of the memory location currently being read from or written to.

Memory data register (MDR)
A register that holds data or an instruction just read from memory, or data waiting to be written to memory.
`Credited: Memory buffer register (MBR)`

Current instruction register (CIR)
A register that holds the instruction currently being decoded and executed.

Accumulator (ACC)
A register that holds the results of calculations and logical operations performed by the ALU.

Address bus
The unidirectional part of the system bus that carries the address of a memory location or I/O device from the CPU.

Data bus
The bidirectional part of the system bus that carries data and instructions between the CPU, memory and I/O devices.

I/O controller
An electronic circuit between the system bus and a peripheral that converts protocols, voltages and data formats and buffers data.
`Credited: Device controller`

Von Neumann architecture
A computer design in which instructions and data are stored together in the same memory and accessed by a single processor over a shared bus.
`Credited: Stored-program concept`

C C# example

`Checking the calculation in C#`

```
int addressBits = 30;
int dataBits = 64;

long locations = 1L << addressBits;            // 2^30 locations
long totalBytes = locations * (dataBits / 8);  // each location holds 8 bytes
double gigabytes = totalBytes / Math.Pow(2, 30);

Console.WriteLine($"Locations: {locations}");
Console.WriteLine($"Maximum memory: {gigabytes} GB");
```

Exam focus

'State' or 'name' questions earn one mark per correct component, so use full names such as memory address register before abbreviating. 'Describe the purpose of' needs what the item holds or does plus a detail: the program counter holds the address of the next instruction and is incremented after each fetch. Questions on the control unit often carry three marks, one for each task, and bus questions usually give one mark per bus for what it carries, with credit for the correct direction. In calculations, show every step in powers of two; method marks are given even if the final figure slips, and a bare answer risks losing them all. Keep bits and bytes separate and divide by 8 at the right point. Common errors include confusing the MAR, which always holds an address, with the MDR, which holds data or an instruction; saying the control bus carries addresses; and calling the address bus two-way. For I/O controllers, examiners credit reasons linked to differences in speed, voltage and protocol between devices. When describing the Von Neumann architecture, mention shared memory for instructions and data, the single shared bus and the resulting bottleneck.

? Check yourself

1. State two tasks carried out by the control unit. [2]
2. Describe the purpose of the memory data register (MDR). [2]
3. Explain why the address bus is unidirectional but the data bus is bidirectional. [2]

Answers: 1. Any two from: fetches instructions from memory, decodes and executes them (fetch–execute cycle), using the ALU as needed (1) / sends control signals to coordinate and monitor hardware so commands are carried out (1) / controls the input and output of data / movement of data into and out of the processor (1) 2. Holds data/an instruction that has just been read (fetched) from memory (1) / or data that is waiting to be written to memory (1) – acts as a buffer between the CPU and memory 3. Only the CPU sends addresses / memory and I/O never need to send an address back, so the address bus only carries signals from CPU to memory/I/O (1) / data must travel both to and from the CPU (reads and writes), so the data bus is two-way (1)